

AiPerm智能权限管理软件 设计说明书

一、软件概述

1.1 软件简介

AiPerm智能权限管理软件是一款基于RBAC（基于角色的访问控制）模型的企业级权限管理系统。系统采用前后端分离的B/S架构，后端基于Spring Boot 4.0 + Java 25构建微服务，前端基于Vue 3 + TypeScript构建单页应用。系统以模块化设计为核心，提供完整的用户认证、权限管理、系统监控、审计日志、文件存储等企业级基础能力，并创新性地融入AI Native设计理念，支持AI辅助的快速业务开发。

1.2 开发背景

随着企业信息化建设的深入推进，权限管理作为企业信息系统的核心基础能力，面临着以下挑战：

- 传统权限系统扩展性差**：传统单体架构的权限系统难以适应微服务和模块化的发展趋势。
- AI时代开发效率要求提升**：AI辅助开发成为趋势，需要权限基座系统能够适配AI协作开发流程。
- 企业级安全合规要求日益严格**：数据安全法、个人信息保护法等法规对权限管理提出了更高的合规要求。
- 多终端多场景适配需求**：企业应用场景日益复杂，需要灵活的权限控制能力。

1.3 设计目标

- 模块化**：采用Spring Modulith实现模块化架构，业务模块按领域划分，松耦合高内聚。
- 安全性**：基于Sa-Token实现完整的认证授权体系，支持多种认证方式和细粒度权限控制。
- 类型安全**：前后端全程类型安全，通过代码生成消除接口调用的类型隐患。
- AI友好**：提供完整的开发技能包和代码生成工具，支持AI辅助快速开发。
- 可扩展**：支持模块热插拔，新增业务模块无需修改核心代码。

二、需求分析

2.1 功能需求

需求编号	需求名称	需求描述	优先级
FR-001	用户认证	支持多种登录方式（密码、短信、邮箱、OAuth）和双因素认证	高
FR-002	用户管理	用户的增删改查、角色分配、部门归属	高
FR-003	角色管理	角色的增删改查、菜单权限分配、数据权限配置	高
FR-004	菜单管理	系统菜单的增删改查、权限标识配置、菜单树管理	高
FR-005	部门管理	组织架构的树形管理、部门层级维护	中

需求编号	需求名称	需求描述	优先级
FR-006	岗位管理	岗位的增删改查、用户岗位分配	中
FR-007	字典管理	字典类型和字典数据的维护管理	中
FR-008	系统监控	在线用户、服务器、缓存、服务监控	中
FR-009	审计日志	操作日志和登录日志的记录与查询	高
FR-010	文件存储	文件上传下载、对象存储管理	中
FR-011	审批流程	审批流程配置和运行管理	中
FR-012	数据仪表盘	系统数据统计和可视化展示	低

2.2 性能需求

需求编号	需求名称	指标要求
PR-001	并发处理	支持500并发用户同时在线操作
PR-002	响应时间	普通查询接口响应时间不超过500ms
PR-003	页面加载	首页加载时间不超过3秒
PR-004	数据容量	支持百万级用户数据管理
PR-005	可用性	系统可用性不低于99.9%

2.3 安全需求

需求编号	需求名称	需求描述
SR-001	身份认证	所有接口必须通过身份认证才能访问
SR-002	权限校验	基于角色的细粒度权限校验，支持按钮级别控制
SR-003	数据安全	敏感数据加密存储，密码使用BCrypt加密
SR-004	操作审计	所有写操作记录审计日志，包含操作人、时间、内容
SR-005	会话管理	Token过期自动失效，支持强制下线
SR-006	SQL注入防护	使用JPA参数化查询，防止SQL注入攻击
SR-007	XSS防护	前端输入内容进行转义处理，防止XSS攻击

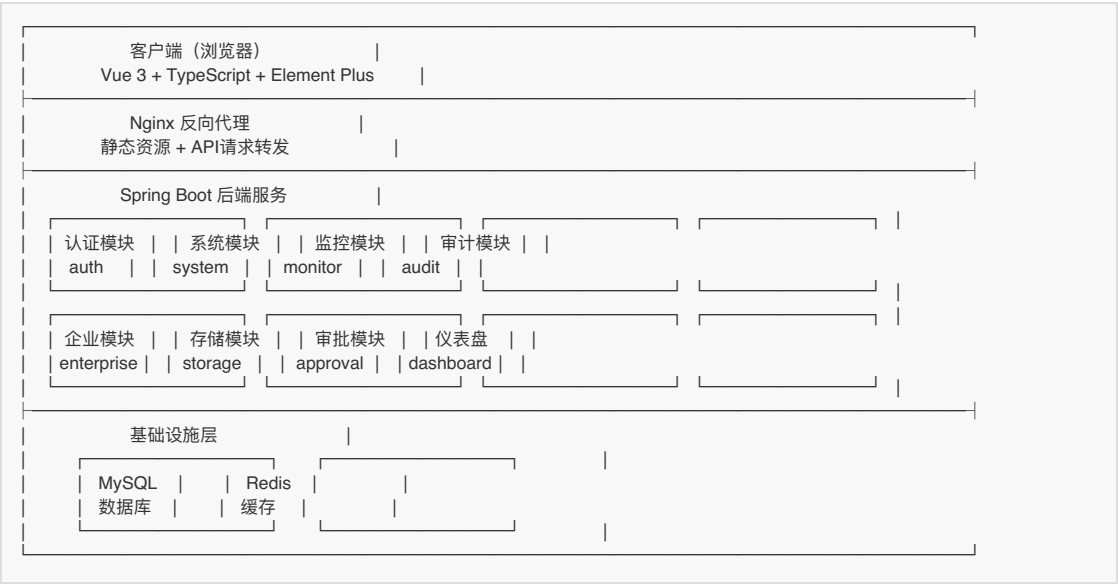
2.4 用户需求

用户角色	需求描述
超级管理员	管理系统所有功能，包括系统配置、用户管理、权限分配等
系统管理员	管理用户、角色、部门等日常管理工作
部门管理员	管理本部门用户和数据
普通用户	使用系统功能，处理日常业务
审计员	查看操作日志和登录日志，进行安全审计

三、总体设计

3.1 系统架构

系统采用经典的前后端分离B/S架构，分为以下层次：



3.2 模块划分

系统按业务领域划分为8个核心模块：

模块名称	模块标识	职责
认证模块	auth	用户认证、登录、令牌管理
系统模块	system	用户、角色、菜单、部门、岗位、字典管理
监控模块	monitor	在线用户、服务、缓存监控
审计模块	audit	操作日志、登录日志
企业模块	enterprise	通知、消息、定时任务
存储模块	storage	文件上传下载、对象存储
审批模块	approval	审批流程配置与运行
仪表盘模块	dashboard	数据统计与可视化

3.3 技术选型

层级	技术	版本	选型理由
后端框架	Spring Boot	4.0.3	成熟稳定的企业级Java框架
模块化	Spring Modulith	2.0.3	Spring官方模块化框架
持久层	Spring Data JPA	-	简化数据访问层开发
权限认证	Sa-Token	1.44.0	轻量级Java权限认证框架
数据库	MySQL	9.x	主流关系型数据库
缓存	Redis	7.x	高性能缓存和会话存储
前端框架	Vue	3.5.x	渐进式前端框架

层级	技术	版本	选型理由
UI组件	Element Plus	2.13.x	成熟的Vue 3 UI组件库
构建工具	Vite	7.x	高性能前端构建工具
类型系统	TypeScript	5.9.x	类型安全的JavaScript超集
API生成	Orval	7.13.x	从Swagger自动生成API客户端

3.4 运行环境

开发环境：

项目	配置
操作系统	macOS / Windows 10+ / Linux
IDE	IntelliJ IDEA / Visual Studio Code
JDK	25
Node.js	18+
包管理	Maven（后端）/ pnpm（前端）

生产环境：

项目	配置
操作系统	Linux（CentOS 7+ / Ubuntu 18.04+）
应用服务器	内嵌Tomcat（Spring Boot）
反向代理	Nginx
数据库	MySQL 8.0+
缓存	Redis 6.0+
JDK	25

四、详细设计

4.1 模块一：认证模块（auth）

4.1.1 模块功能

认证模块负责用户的身份验证和令牌管理，支持以下认证方式： - 用户名密码登录 - 手机短信验证码登录 - 邮箱验证码登录 - OAuth2.0第三方登录 - 双因素认证（2FA）

4.1.2 模块接口

接口	方法	路径	描述
密码登录	POST	/auth/login/password	用户名密码登录
短信登录	POST	/auth/login/sms	手机验证码登录
邮箱登录	POST	/auth/login/email	邮箱验证码登录
退出登录	POST	/auth/logout	退出登录
获取验证码	GET	/auth/captcha	获取图形验证码

接口	方法	路径	描述
发送短信	POST	/auth/sms/send	发送短信验证码

4.1.3 处理流程

用户输入凭据 → 验证图形验证码 → 验证用户凭据 → 检查账号状态 → 生成Token → 检查2FA → 返回Token信息 → 前端存储Token → 跳转首页

4.2 模块二：系统模块（system）

4.2.1 模块功能

系统模块是核心业务模块，包含用户管理、角色管理、菜单管理、部门管理、岗位管理和字典管理六个子模块。

用户管理： - 用户的增删改查 - 用户角色分配 - 用户部门归属 - 密码重置 - 用户状态管理

角色管理： - 角色的增删改查 - 菜单权限分配 - 数据权限配置 - 角色状态管理

菜单管理： - 菜单的增删改查 - 菜单树形结构管理 - 权限标识配置 - 菜单类型（目录/菜单/按钮）

4.2.2 模块接口

接口	方法	路径	描述
用户列表	GET	/system/user	分页查询用户列表
用户详情	GET	/system/user/{id}	获取用户详情
创建用户	POST	/system/user	创建新用户
更新用户	PUT	/system/user	更新用户信息
删除用户	DELETE	/system/user/{id}	删除用户
角色列表	GET	/system/role	分页查询角色列表
菜单列表	GET	/system/menu	查询菜单树
部门树	GET	/system/dept/tree	查询部门树

4.2.3 处理流程

权限校验流程：

用户请求 → Sa-Token拦截器 → 检查登录状态(@SaCheckLogin) → 检查权限标识(@SaCheckPermission) → 通过/拒绝
--

数据权限过滤流程：

查询请求 → 获取当前用户角色 → 获取数据权限范围 → 构建过滤条件 → 执行查询 → 返回过滤后的数据
--

4.3 模块三：监控模块（monitor）

4.3.1 模块功能

监控模块提供系统运行状态的实时监控能力，包含在线用户监控、服务器监控、缓存监控和服务监控。

4.3.2 模块接口

接口	方法	路径	描述
在线用户	GET	/monitor/online	查看在线用户列表
强制下线	DELETE	/monitor/online/{token}	强制用户下线
服务器信息	GET	/monitor/server	获取服务器状态
缓存信息	GET	/monitor/cache	获取缓存状态
清理缓存	DELETE	/monitor/cache	清理缓存

4.3.3 处理流程

服务器监控数据采集流程：

定时采集 → CPU使用率 → 内存使用率 → 磁盘使用率
→ JVM信息 → 线程信息 → 组装返回数据

4.4 模块四：审计模块（audit）

4.4.1 模块功能

审计模块通过AOP切面自动记录系统操作日志和登录日志，支持日志查询和导出。

4.4.2 模块接口

接口	方法	路径	描述
操作日志	GET	/audit/operation	分页查询操作日志
登录日志	GET	/audit/login	分页查询登录日志

4.4.3 处理流程

操作执行 → @Log注解切面拦截 → 记录操作信息
→ 异步写入数据库 → 操作完成返回

五、数据结构设计

5.1 数据存储

系统使用MySQL作为主数据库，Redis作为缓存数据库。

MySQL：存储所有业务数据，包括用户、角色、权限、日志等。**Redis**：存储会话Token、缓存数据、验证码等临时数据。

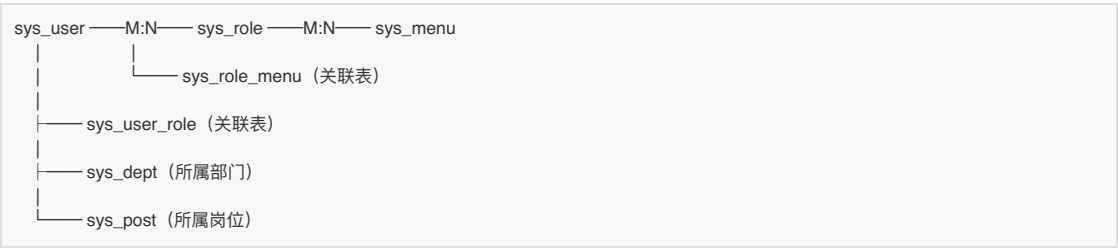
5.2 数据格式

主要数据表结构：

表名	描述	主要字段
sys_user	用户表	id, username, password, phone, email, status, dept_id
sys_role	角色表	id, name, code, status, data_scope

表名	描述	主要字段
sys_menu	菜单表	id, name, path, permission, type, parent_id
sys_dept	部门表	id, name, parent_id, sort, status
sys_post	岗位表	id, name, code, sort, status
sys_dict_type	字典类型表	id, name, code, status
sys_dict_data	字典数据表	id, type_id, label, value, sort
sys_user_role	用户角色关联表	id, user_id, role_id
sys_role_menu	角色菜单关联表	id, role_id, menu_id
sys_operation_log	操作日志表	id, user_id, module, operation, method, params
sys_login_log	登录日志表	id, username, ip, location, browser, os

5.3 数据关系



六、接口设计

6.1 内部接口

系统内部各模块之间通过Spring事件机制进行通信，使用Spring Modulith的事件发布/订阅模式实现模块间解耦。

6.2 外部接口

RESTful API规范：

规范项	说明
协议	HTTPS
请求格式	JSON
响应格式	JSON
认证方式	Bearer Token（Sa-Token）
字符编码	UTF-8

统一响应格式：

```
{
  "code": 200,
  "message": "success",
  "data": {}
}
```

分页响应格式：

```
{
  "code": 200,
  "message": "success",
  "data": {
    "total": 100,
    "list": [],
    "pageNum": 1,
    "pageSize": 10,
    "pages": 10
  }
}
```

6.3 接口规范

1. 所有接口必须添加 @SaCheckLogin 注解，确保用户已登录。
2. 写操作（POST/PUT/DELETE）必须添加 @SaCheckPermission 注解进行权限校验。
3. 写操作必须添加 @Log 注解记录审计日志。
4. 分页查询使用统一的 PageResult<T> 返回格式。
5. 接口路径遵循 RESTful 风格，使用小写字母和连字符。

七、算法设计

7.1 核心算法

RBAC权限树构建算法：

系统采用递归算法从数据库读取菜单数据，构建树形权限结构。算法步骤如下： 1. 查询所有有效菜单记录。 2. 以parent_id为0的记录作为根节点。 3. 递归查找每个节点的子节点。 4. 构建完整的权限树。

数据权限过滤算法：

根据角色的数据权限范围构建SQL过滤条件： - 全部数据：不添加过滤条件。 - 本部门数据：过滤 dept_id等于用户部门。 - 本部门及以下：使用部门树过滤。 - 自定义：使用指定的部门列表过滤。

7.2 算法流程

权限校验流程：

1. 从Token中获取用户ID
2. 查询用户拥有的所有角色
3. 合并所有角色的权限标识集合
4. 检查目标权限标识是否在集合中
5. 返回校验结果（通过/拒绝）

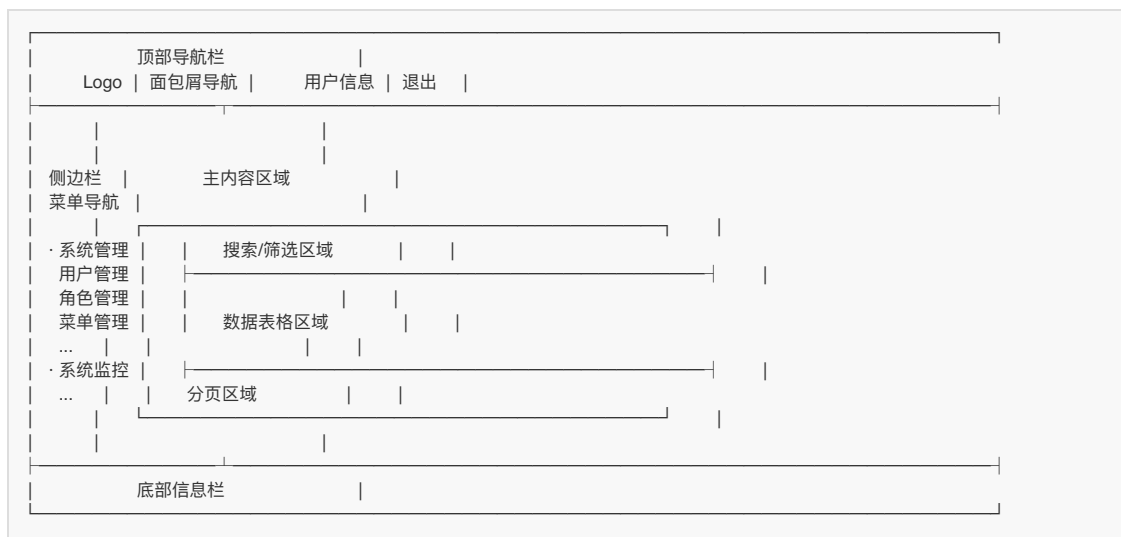
7.3 算法复杂度

- 权限树构建：O(n)，n为菜单总数
- 权限校验：O(1)，使用HashSet存储权限标识
- 数据权限过滤：O(n)，n为部门数量

八、界面设计

8.1 界面布局

系统采用经典的后台管理界面布局：



8.2 交互设计

1. **列表页交互**：表格展示数据，支持排序、筛选、分页，操作列包含编辑、删除等按钮。
2. **表单交互**：新增和编辑使用弹窗（Dialog）形式，表单验证通过后提交。
3. **树形交互**：部门和菜单管理使用树形控件，支持展开/折叠、拖拽排序。
4. **消息提示**：操作成功/失败通过Message组件提示，确认操作使用MessageBox。

8.3 界面元素

系统使用Element Plus组件库，主要使用以下组件： - ETable：数据表格 - EForm：表单 - EDialog：弹窗 - ETree：树形控件 - EMenu：导航菜单 - EIPagination：分页 - EIMessage：消息提示 - EIMessageBox：确认对话框

九、安全设计

9.1 安全机制

1. **认证机制**：基于Sa-Token的Token认证，Token存储在Redis中，支持过期自动清理。
2. **授权机制**：基于RBAC模型的权限控制，支持菜单权限和数据权限。
3. **密码安全**：使用BCrypt算法加密存储密码，防止彩虹表攻击。
4. **请求防护**：使用Spring Security防护CSRF攻击。

9.2 数据保护

1. **传输加密**：生产环境使用HTTPS加密传输。
2. **存储加密**：敏感字段（密码）加密存储。
3. **逻辑删除**：业务数据采用逻辑删除，保留数据可恢复性。
4. **SQL注入防护**：使用JPA参数化查询，杜绝SQL注入。

9.3 访问控制

1. **接口级别**：通过 @SaCheckLogin 和 @SaCheckPermission 注解控制接口访问。
2. **按钮级别**：前端通过权限指令 v-hasPermi 控制按钮显示。
3. **数据级别**：通过数据权限过滤控制用户可见数据范围。
4. **审计追踪**：所有敏感操作记录审计日志。

十、测试设计

10.1 测试策略

测试类型	测试范围	测试工具
单元测试	Service层业务逻辑	JUnit 5 + Mockito
集成测试	API接口联调	Spring Boot Test
前端测试	组件功能测试	Vitest
E2E测试	完整业务流程	手工测试
性能测试	并发和响应时间	JMeter

10.2 测试用例

用例编号	用例名称	前置条件	操作步骤	预期结果
TC-001	正常登录	用户已注册	输入正确用户名密码 点击登录	登录成功跳转首页
TC-002	密码错误	用户已注册	输入错误密码点击登 录	提示密码错误
TC-003	创建用户	管理员已登录	填写用户信息点击保 存	用户创建成功
TC-004	权限校验	用户无某权限	访问需权限的接口	提示无权限
TC-005	文件上传	用户已登录	选择文件点击上传	文件上传成功

10.3 测试环境

项目	配置
操作系统	macOS / Linux
JDK	25
Node.js	18+
数据库	MySQL 8.0+
缓存	Redis 6.0+
浏览器	Chrome 最新版